Chapter 30: Polynomials and the FFT

1) Fourier analysis allows us to express a signal as a weighted sum of phase-shifted sinusoids of varying frequencies. The weights and phases associated with the frequencies characterize the signal in the frequency domain.

2) Polynomials:

$$A(x) = \sum_{j=0}^{n-1} a_j x^j$$

→ $A(x)$ is said to have degree k if the highest non-zero coefficient of $A(x)$ is a_k.

→ Any integer strictly greater than the degree of a polynomial is a "degree-bound" of that polynomial. The degree of a polynomial of degree-bound n may be any integer between 0 and $n-1$, inclusive.

Polynomial addition: If $A(x)$ and $B(x)$ are polynomials of degree-bound n, their sum $C(x)$ is a polynomial of degree-bound n such that $C(x) = A(x) + B(x)$ for all x in the underlying field.

$$A(x) = \sum_{j=0}^{n-1} a_j x^j \quad \text{and} \quad B(x) = \sum_{j=0}^{n-1} b_j x^j$$

$$C(x) = \sum_{j=0}^{n-1} c_j x^j = \sum_{j=0}^{n-1} (a_j + b_j) x^j \quad \text{i.e. } c_j = a_j + b_j \quad \forall j \in [1, n-1]$$

Polynomial multiplication: If $A(x)$ and $B(x)$ are polynomials of degree-bound n, we say that their product $C(x)$ is a polynomial of degree-bound $2n-1$ such that $C(x) = A(x)B(x)$ for all x in the underlying field.

$$C(x) = \sum_{j=0}^{2n-2} c_j x^j \quad \text{where} \quad c_j = \sum_{k=0}^{j} a_k b_{j-k}$$

$$\text{degree}(C) = \text{degree}(A) + \text{degree}(B), \text{ implying}$$

$$\begin{aligned}\text{degree-bound}(C) &= \text{degree-bound}(A) + \text{degree-bound}(B) - 1 \\ &\leq \text{degree-bound}(A) + \text{degree-bound}(B)\end{aligned}$$

→ We shall speak of the degree-bound of C as being the sum of the degree-bounds of A and B, since if a polynomial has degree-bound k, it also has degree-bound $k+1$.

3) Representation of polynomials:

We shall study 2 representations of polynomials: coefficient representation and point-value representation. The two representations are in a sense equivalent: a polynomial in point-value form has a unique counterpart in coefficient form.

→ Coefficient representation: A coefficient representation of a polynomial $A(x) = \sum_{j=0}^{n-1} a_j x^j$ of degree-bound n is a vector of coefficients $a = (a_0, a_1, a_2 \ldots a_{n-1})$.

Horner's rule for evaluating a polynomial:

$A(x)$ can be evaluated at x_0 by

$$A(x_0) = a_0 + x_0(a_1 + x_0(a_2 + \cdots + x_0(a_{n-2} + x_0(a_{n-1}))\cdots))$$

Evaluating a polynomial using Horner's rule takes $\Theta(n)$ time.

Polynomial addition using coefficient representation of polynomials takes $\Theta(n)$ time:

$a = (a_0, a_1, \ldots a_{n-1})$ and $b = (b_0, b_1, \ldots b_{n-1})$

then $c = (c_0, c_1, \ldots c_{n-1})$ where $c_j = a_j + b_j \quad \forall j \in [1, n-1]$

Straight forward multiplication of two degree-bound n polynomials $A(x)$ and $B(x)$ represented in coefficient form takes time $\Theta(n^2)$, since each coefficient in vector "a" must be multiplied with each coefficient in vector "b". The resulting vector c is called the "convolution" of vectors a and b denoted by $c = a \otimes b$.

→ Point-value representation:

A point-value representation of a polynomial $A(x)$ of degree-bound n is a set of n point-value pairs: $\{(x_0, y_0), (x_1, y_1), (x_2, y_2) \ldots (x_{n-1}, y_{n-1})\}$

such that all x_k are distinct and $y_k = A(x_k)$.

A polynomial has many different point-value representations, since any set of n distinct points $x_0, x_1, x_2 \ldots x_{n-1}$ can be used as a basis for the representation.

→ Converting a coefficient representation of a polynomial to a point-value representation provided a set of n distinct points $x_0, x_1 \ldots x_{n-1}$ is for such involves evaluating the polynomial at $x_0, x_1 \ldots x_{n-1}$ using Horner's method this n-point evaluation

Evaluating a polynomial using Horner's rule takes $\Theta(n)$ time.

Polynomial addition using coefficient representation of polynomials takes $\Theta(n)$ time :

$a = (a_0, a_1, \ldots a_{n-1})$ and $b = (b_0, b_1, \ldots b_{n-1})$

then $c = (c_0, c_1, \ldots c_{n-1})$ where $c_j = a_j + b_j \quad \forall j \in [1, n-1]$

Straight forward multiplication of two degree-bound n polynomials $A(x)$ and $B(x)$ represented in coefficient form takes time $\Theta(n^2)$, since each coefficient in vector "a" must be multiplied with each coefficient in vector "b". The resulting vector c is called the "convolution" of vectors a and b denoted by $c = a \otimes b$.

→ Point-value representation :

A point-value representation of a polynomial $A(x)$ of degree-bound n is a set of n point-value pairs: $\{(x_0, y_0), (x_1, y_1), (x_2, y_2) \ldots (x_{n-1}, y_{n-1})\}$

such that all x_k are distinct and $y_k = A(x_k)$.

A polynomial has many different point-value representations, since any set of n distinct points $x_0, x_1, x_2 \ldots x_{n-1}$ can be used as a basis for the representation.

→ Converting a coefficient representation of a polynomial to a point-value representation provided a set of n distinct points $x_0, x_1 \ldots x_{n-1}$ is known involves evaluating the polynomial at $x_0, x_1 \ldots x_{n-1}$ using Horner's method this n-point evaluation

takes time $\Theta(n^2)$.

→ The inverse of evaluation is called interpolation and comprises of determining the coefficient form of a polynomial provided a point-value representation. Interpolation is well defined, assuring that the degree-bound of the interpolating polynomial equals the number of given point value pairs.

Theorem (Uniqueness of interpolating polynomial)

For any set $\{(x_0, y_0), (x_1, y_1), \ldots, (x_{n-1}, y_{n-1})\}$ of n point value pairs such that all the x_k values are distinct, there is a unique polynomial $A(x)$ of degree-bound n such that $y_k = A(x_k)$ for $k = 0, 1, \ldots, n-1$.

Proof: Eq^n. $y_k = A(x_k)$ is equivalent to :-

$$\begin{pmatrix} 1 & x_0 & x_0^2 & \cdots & x_0^{n-1} \\ 1 & x_1 & x_1^2 & \cdots & x_1^{n-1} \\ \vdots & & & & \\ 1 & x_{n-1} & x_{n-1}^2 & \cdots & x_{n-1}^{n-1} \end{pmatrix} \begin{pmatrix} a_0 \\ a_1 \\ \vdots \\ a_{n-1} \end{pmatrix} = \begin{pmatrix} y_0 \\ y_1 \\ \vdots \\ y_{n-1} \end{pmatrix}$$

The matrix on the left is denoted $V(x_0, x_1, x_2, \ldots, x_{n-1})$ and is called the Vander-monde matrix.

The determinant of this matrix is :-

$$\prod_{0 \le j < k \le n-1} (x_k - x_j)$$

Since the x_i's are distinct, the determinant is non-zero. A square matrix with non-zero determinant is always nonsingular (invertible).

Hence, the vector "a" can be uniquely found out

as:-

$$a = V(x_0, x_1, \ldots x_{n-1})^{-1} Y$$

Using LUP decomposition of the Vandermonde matrix we can solve for a in $O(n^3)$ time.

→ A faster algorithm for n-point interpolation is based upon Lagrange's formula:-

$$A(x) = \sum_{k=0}^{n-1} y_k \frac{\prod_{j \neq k} (x - x_j)}{\prod_{j \neq k} (x_k - x_j)}$$

You can compute the coefficients of $A(x)$ using Lagrange's formula in time $\Theta(n^2)$. Thus, n-point evaluation and interpolation are well-defined inverse operations that transform between the coefficient representation of a polynomial and point-value representation. The algorithms to do so take time $\Theta(n^2)$.

→ **Polynomial addition using point-value representation:** If $C(x) = A(x) + B(x)$ then $C(x_k) = A(x_k) + B(x_k)$ for any point x_k.

If the point-value representation of A is:

$$\{(x_0, y_0), (x_1, y_1), \ldots (x_{n-1}, y_{n-1})\}$$

and the point-value representation of B is:

$$\{(x_0, y_0'), (x_1, y_1'), \ldots (x_{n-1}, y_{n-1}')\}.$$

then the point-value representation of C is:-

$$\{(x_0, y_0 + y_0'), (x_1, y_1 + y_1'), \ldots (x_{n-1}, y_{n-1} + y_{n-1}')\}$$

The time to add 2 polynomials of degree-bound n is $\Theta(n)$.

→ **Polynomial multiplication using point-value representation:** If $C(x) = A(x)B(x)$ then $C(x_k) = A(x_k)B(x_k)$ for any point x_k i.e. We can pointwise multiply a point-value representation for A by a point-value representation for B to obtain a point-value representation for C.

If A and B are polynomials of degree-bound n, C would be a polynomial of degree-bound $2n$, i.e. We need $2n$ point-value pairs for representing C. We must therefore begin with an extended point-value representation for A and B consisting of $2n$ point-value pairs each.

Given an extended point-value representation for A:-

$\{(x_0, y_0), (x_1, y_1) \ldots\ldots (x_{2n-1}, y_{2n-1})\}$

and an extended point-value representation for B:-

$\{(x_0, y_0'), (x_1, y_1') \ldots\ldots (x_{2n-1}, y_{2n-1}')\}$

then a point-value representation for C is:-

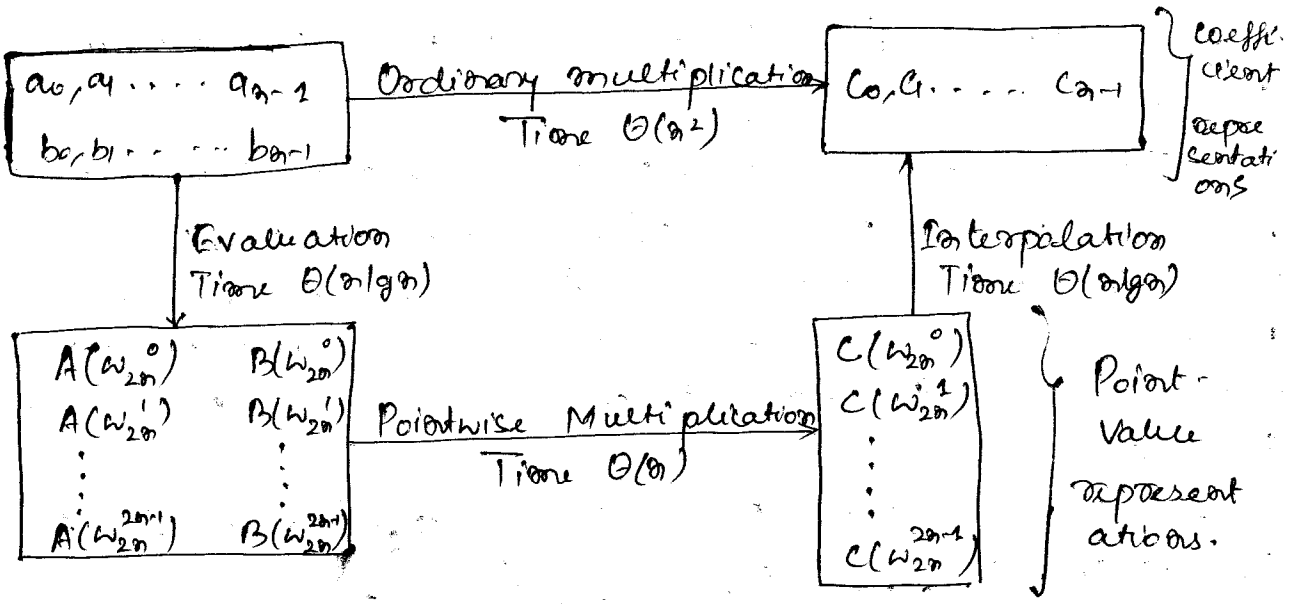
$\{(x_0, y_0 y_0'), (x_1, y_1 y_1'), (x_2, y_2 y_2') \ldots\ldots (x_{2n-1}, y_{2n-1} y_{2n-1}')\}$

Thus, multiplication of two polynomials in point-value form is $\Theta(n)$.

→ To evaluate a polynomial in point-value form at a given point, we first need to convert it into coefficient form.

Fast multiplication of polynomials in coefficient form: We can use the linear-time multiplication method for polynomials in point-value form to expedite polynomial multiplication in coefficient form. All we need is the ability to

convert a polynomial quickly from coefficient form to point-value form (evaluate) and vice-versa (interpolate).



By choosing the $2n$ "complex $2n$th roots of unity" we can perform evaluation and interpolation in time $\Theta(n \lg n)$. We produce a point-value representation by taking the DFT of a coefficient vector. We produce a coefficient vector by taking inverse-DFT of point-value pairs. FFT performs DFT and inverse-DFT in $\Theta(n \lg n)$ time.

→ We have the following $\Theta(n \lg n)$ time procedure for multiplying two polynomials $A(x)$ and $B(x)$ of degree-bound n, where the input and output representations are in coefficient form. We assume n is a power of 2; this requirement can always be met by adding high-order zero coefficients.

1) Double-degree-bound: Create coefficient representations of $A(x)$ and $B(x)$ as degree-bound $2n$ polynomials

by adding n high-order zero coefficients to each.
Time: $\Theta(n)$

2) Evaluate: Compute point-value representations of $A(x)$ and $B(x)$ of length $2n$, by evaluating at $2n$ complex $2n$th roots of unity. Time: $\Theta(n \lg n)$.

3) Pointwise multiply: Pointwise multiply $A(x)$ and $B(x)$ to obtain $2n$ point-value pairs for $C(x)$.

Time: $\Theta(n)$.

4) Interpolate: Obtain a coefficient representation of $C(x)$ by application of inverse-DFT.

Time: $\Theta(n \lg n)$

Theorem

The product of 2 polynomials of degree-bound n can be computed in time $\Theta(n \lg n)$, with both the input and output representations in coefficient form.

4)

→ **Complex roots of unity:** A "complex nth root of unity" is a complex no. w such that $w^n = 1$. There are exactly n complex nth roots of unity.

$w_n = e^{\frac{2\pi}{n} i}$ is called the "principal nth root of unity". All the other complex nth roots of unity are powers of w_n. Thus, the n complex nth roots of unity are: $w_n^0, w_n^1, w_n^2, \ldots w_n^{n-1}$.

$w_n^k = w_n^{k \bmod n}$ eg: $w_n^n = w_n^0 = 1$, $w_n^{n+1} = w_n^1$

$w_n^{n-1} = w_n^{-1}$ $[= w_n^n \cdot w_n^{-1} = w_n^{-1}]$

Since $w_n^k = w_n^{an+b}$ where $k = an+b$

$= w_n^{an} \cdot w_n^b = (w_n^n)^a \cdot w_n^b = (w_n^0)^a \cdot w_n^b = w_n^b$

Cancellation lemma : For any integers $n \geq 0$, $k \geq 0$ and $d > 0$ we have

$$W_{dn}^{dk} = W_n^k$$

Proof: $W_{dn}^{dk} = \left(e^{\frac{2\pi i}{dn}}\right)^{dk} = \left(e^{\frac{2\pi i}{n}}\right)^k = W_n^k$

→ In signal processing applications W_n is defined as $W_n = e^{-\frac{2\pi}{n} i} = \cos\left(\frac{2\pi}{n}\right) - i \sin(2\pi/n)$. The underlying mathematics remains the same

Halving lemma : If $n > 0$ is even, then the squares of the n complex nth roots of unity are the $n/2$ complex $n/2$th roots of unity.

Proof: By cancellation lemma : $(W_n^k)^2 = W_{n/2}^k$ for $k \geq 0$. Note that if we square all of the complex nth roots of unity, then each $n/2$th root of unity is obtained exactly twice, since :-

$$(W_n^{k+n/2})^2 = W_n^{2k+n} = W_n^{2k} \cdot W_n^n = W_n^{2k} = (W_n^k)^2$$

Thus, square of $W_n^{n/2+k}$ is same as that of W_n^k.

Summation lemma : For any integer $n \geq 1$ and non-negative integer k not divisible by n

$$\sum_{j=0}^{n-1} (W_n^k)^j = 0$$

Proof :- $\sum_{j=0}^{n-1} (W_n^k)^j = \frac{(W_n^k)^n - 1}{W_n^k - 1} = \frac{(W_n^n)^k - 1}{W_n^k - 1}$

$= \frac{1^K - 1}{\omega_n^K - 1} = \frac{0}{\omega_n^K - 1} = 0$ if K is not a multiple of n.

5) The DFT

$$A(x) = \sum_{j=0}^{n-1} a_j x^j$$

We wish to evaluate $A(x)$ at $\omega_n^0, \omega_n^1, \omega_n^2 \ldots \omega_n^{n-1}$

$$\text{Let } y_k = A(\omega_n^k) = \sum_{j=0}^{n-1} a_j (\omega_n^k)^j$$

The vector $y = (y_0, y_1, y_2 \ldots y_{n-1})$ is called the "Discrete Fourier Transform" of the coefficient vector $a = (a_0, a_1, a_2 \ldots a_{n-1})$. We write $y = DFT_n(a)$.

6) The FFT

Fast Fourier Transform provides a method to compute $DFT_n(a)$ in time $\Theta(n \lg n)$.

Let us define :-

$$A^{[0]}(x) = a_0 + a_2 x + a_4 x^2 + \ldots + a_{n-2} x^{n/2-1}$$

$$A^{[1]}(x) = a_1 + a_3 x + a_5 x^3 + \ldots + a_{n-1} x^{n/2-1}$$

Then, $A(x) = A^{[0]}(x^2) + x A^{[1]}(x^2)$.

The problem of evaluating $A(x)$ at $\omega_n^0, \omega_n^1 \ldots \omega_n^{n-1}$ reduces to :-

(i) evaluating $A^{[0]}(x)$ and $A^{[1]}(x)$ at $(\omega_n^0)^2, (\omega_n^1)^2, \ldots (\omega_n^{n-1})^2$

and then

(ii) combining the results using $A(x) = A^{[0]}(x^2) + x A^{[1]}(x^2)$

$(\omega_n^0)^2, (\omega_n^1)^2, \ldots (\omega_n^{n-1})^2$ are actually the $n/2$ complex $n/2$ th roots of unity with each root occurring twice.

i.e. we evaluate the polynomials $A^{[0]}$ and $A^{[1]}$ at the $n/2$ complex $n/2$ th roots of unity. We thus have divided an n-element DFT$_n$ computation into two $n/2$-element DFT$_{n/2}$ computations.

RECURSIVE_FFT(a)

```
1   n ← length[a]                    ▷ n is a power of 2
2   if n = 1
3       then return a
4   ω_n ← e^{2πi/n}
5   ω ← 1                            ▷ ω is initialized with ω_n^0
6   a^[0] ← (a_0, a_2, a_4 .... a_{n-2})
7   a^[1] ← (a_1, a_3, a_5 .... a_{n-1})
8   Y^[0] ← RECURSIVE_FFT(a^[0])
9   Y^[1] ← RECURSIVE_FFT(a^[1])
10  for k ← 0 to n/2 - 1
11      do Y_k ← Y_k^[0] + ω Y_k^[1]
12         Y_{k+n/2} ← Y_k^[0] - ω Y_k^[1]
13         ω ← ω ω_n
14  return Y
```

$$Y_{k+n/2} = Y_k^{[0]} - \omega_n^k Y_k^{[1]} = Y_k^{[0]} + \omega_n^{n/2} \cdot \omega_n^k Y_k^{[1]}$$

$$= Y_k^{[0]} + \omega_n^{k+n/2} Y_k^{[1]}$$

$$= A^{[0]}(\omega_n^{2k}) + \omega_n^{k+n/2} A^{[1]}(\omega_n^{2k})$$

$$= A^{[0]}(\omega_n^{2k+n}) + \omega_n^{k+n/2} A^{[1]}(\omega_n^{2k+n})$$

$= A(\omega_n^{k+n/2})$

→ Exclusive of the recursive calls, each invocation of RECURSIVE-FFT takes time $\Theta(n)$. Thus :-

$$T(n) = 2T(n/2) + \Theta(n)$$
$$= \Theta(n \lg n)$$